JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Gaussian Splatting for Novel View Synthesis

Prof. Ziyun (Claude) Wang
Department of Electrical and Computer Engineering
Spring 2026

LABORATORY FOR Computational Sensing + Robotics
Data Science and AI Institute
JHU Electrical and Computer Engineering

Logistics

- Late day policy
- HW2 out today, due in 2 weeks
- Guest Lecture on tactile information on Wednesday

Recap: Stereo

- 3D points from two views – “reconstruction.”
- Is it enough to just see points?
  - If not, how do we render it?

Depth to Point Cloud

Step 1: pixel $\rightarrow$ normalized coordinates

$$\boxed{x_n = \frac{u - c_x}{f_x}, \qquad y_n = \frac{v - c_y}{f_y}}$$

Step 2: (normalized, depth) $\rightarrow$ 3D point in camera frame

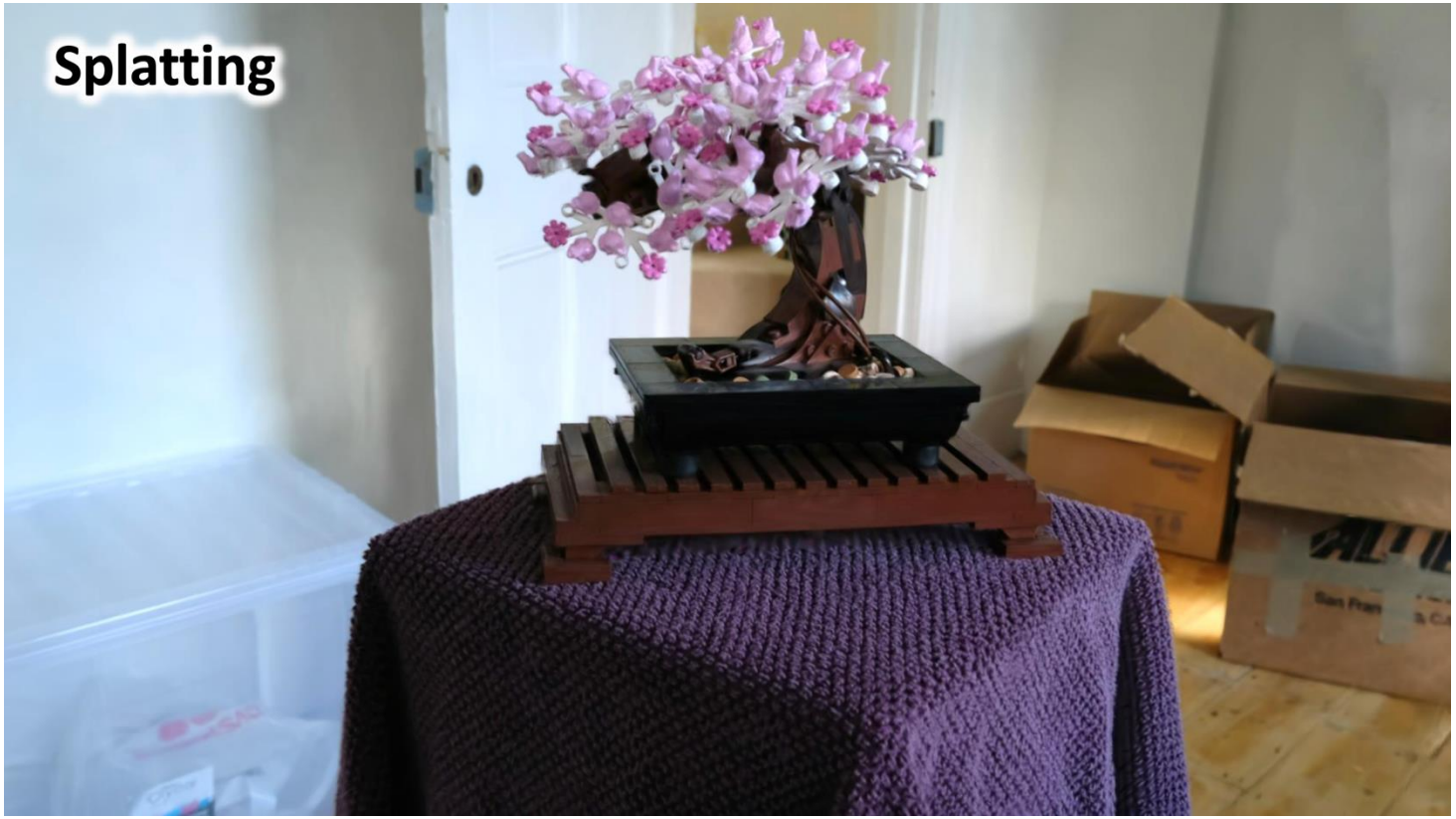
$$\boxed{(X, Y, Z)^\top = z\,(x_n, y_n, 1)^\top \;\Rightarrow\; X = z\,x_n,\; Y = z\,y_n,\; Z = z}$$

Gaussian Splatting

Viewing 3D points from a new view

Sparse, unrealistic lighting

View Synthesis

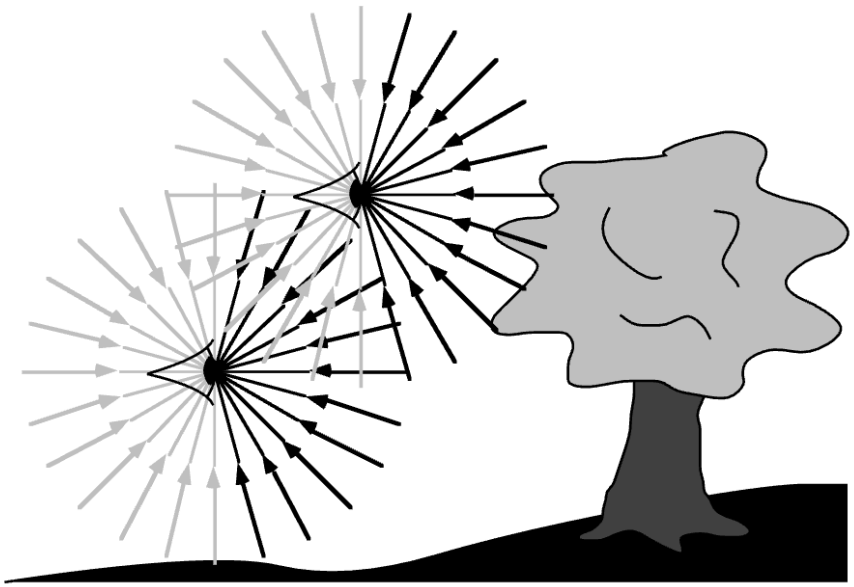


Novel View Synthesis

https://scannetpp.mlsg.cit.tum.de/scannetpp/benchmark/nvs_iphone

Radiance Field

- Modeling "Radiance"



[Adelson & Bergen 91']

The function describes the information available to an observer at any point in space and time

"Everything you need for seeing"

Neural Radiance Field

A **neural radiance field** represents a scene as a continuous function implemented by an MLP:

$$f_\theta : (\boldsymbol{x} \in \mathbb{R}^3,\ \boldsymbol{d} \in \mathbb{R}^3) \mapsto \big(\sigma(\boldsymbol{x}),\ \boldsymbol{c}(\boldsymbol{x}, \boldsymbol{d})\big).$$

- $\boldsymbol{x}$: 3D location (usually in world coordinates)
- $\boldsymbol{d}$: viewing direction (unit vector)
- $\sigma(\boldsymbol{x}) \geq 0$: **volume density** (how much matter is present; controls occlusion)
- $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{d}) \in \mathbb{R}^3$: **radiance/color** emitted toward direction $\boldsymbol{d}$

Modeling Neural Radiance Field

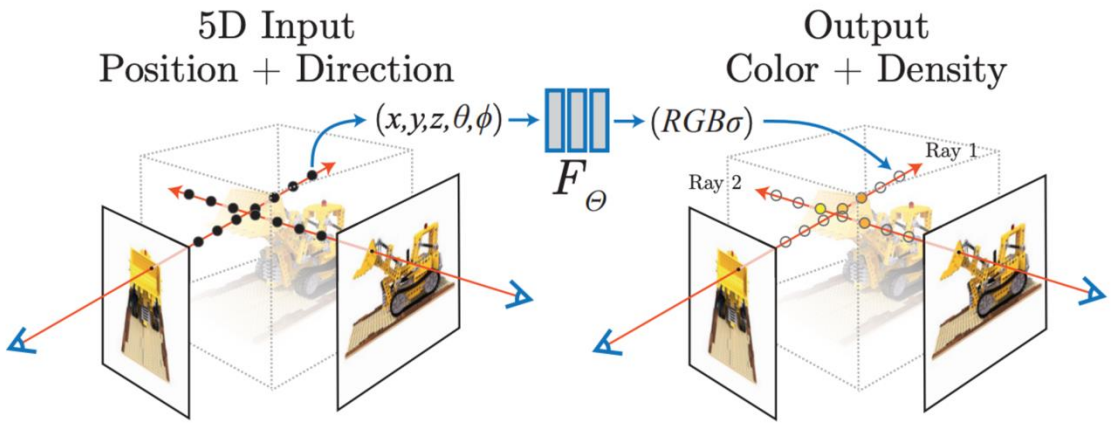
Given a set of sparse views of an object with known camera poses

Optimize a NeRF model

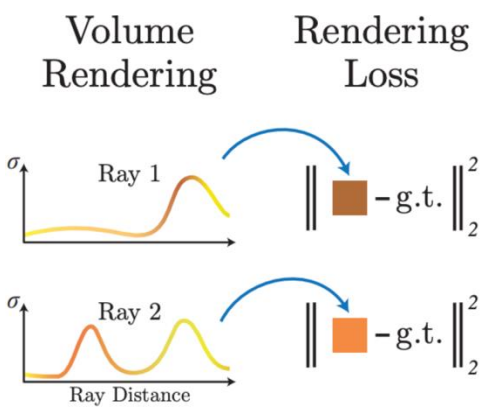
3D reconstruction viewable from any angle

Neural Radiance Field (NeRF)

Render



Fit NeRF



NeRF Results

Zip-NeRF, Barron et al., ICCV 2023

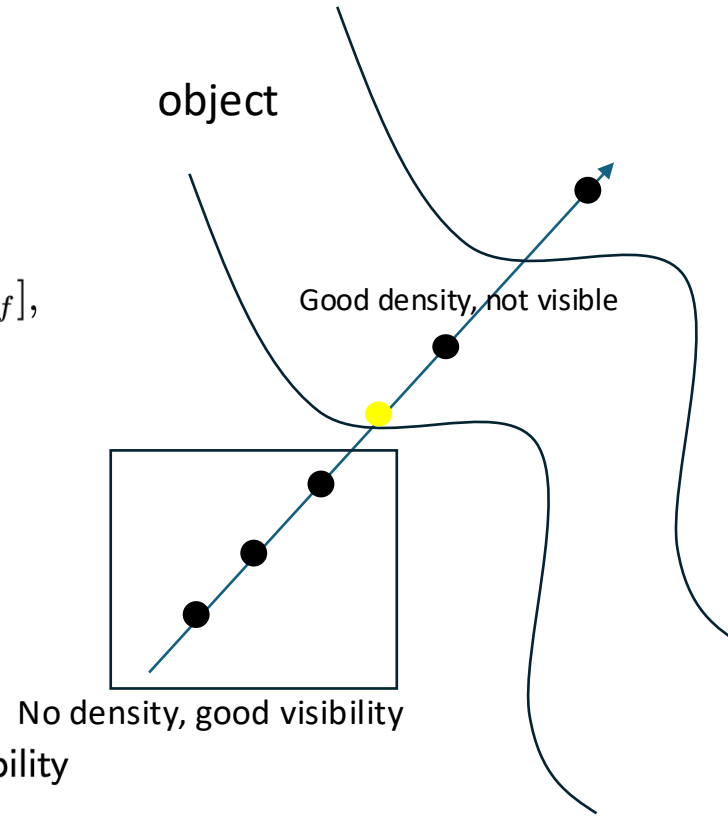
Volume Rendering

For a camera ray parameterized as

$$\boldsymbol{x}(t) = \boldsymbol{o} + t\,\boldsymbol{d}, \qquad t \in [t_n, t_f],$$

$$\boldsymbol{C}(\boldsymbol{o}, \boldsymbol{d}) = \int_{t_n}^{t_f} T(t)\,\sigma(\boldsymbol{x}(t))\,\boldsymbol{c}(\boldsymbol{x}(t), \boldsymbol{d})\,dt$$

$$\boxed{T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\boldsymbol{x}(s))\,ds\right)}$$



- $T(t)$: visibility / accumulated "not yet occluded" probability
- $\sigma(\boldsymbol{x})$: density (controls how quickly visibility decays)
- $\boldsymbol{c}(\boldsymbol{x}, \boldsymbol{d})$: radiance/color contributed at location $\boldsymbol{x}$ toward view direction $\boldsymbol{d}$

Discretize along the ray

In practice, NeRF uses a sampled approximation with points $\{t_k\}_{k=1}^{K}$ and step sizes $\delta_k = t_{k+1} - t_k$. Let

$$\sigma_k = \sigma(\boldsymbol{x}(t_k)), \qquad \boldsymbol{c}_k = \boldsymbol{c}(\boldsymbol{x}(t_k), \boldsymbol{d}).$$

Define per-sample opacity and weights:

$$\boxed{\alpha_k = 1 - \exp(-\sigma_k \, \delta_k),} \qquad T_k = \prod_{j<k} (1 - \alpha_j), \qquad w_k = T_k \, \alpha_k.$$

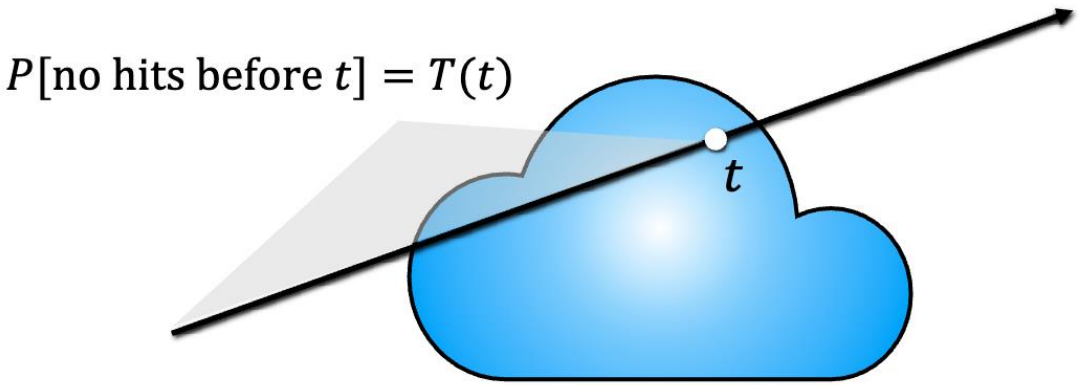
Assume constant rate (%) of decay, how much light is absorbed

Then the pixel color is

$$\boxed{\boldsymbol{C} \approx \sum_{k=1}^{K} w_k \, \boldsymbol{c}_k \; + \; T_{K+1} \, \boldsymbol{C}_{\text{bg}}}.$$

We sometimes ignore the background

Volume Rendering

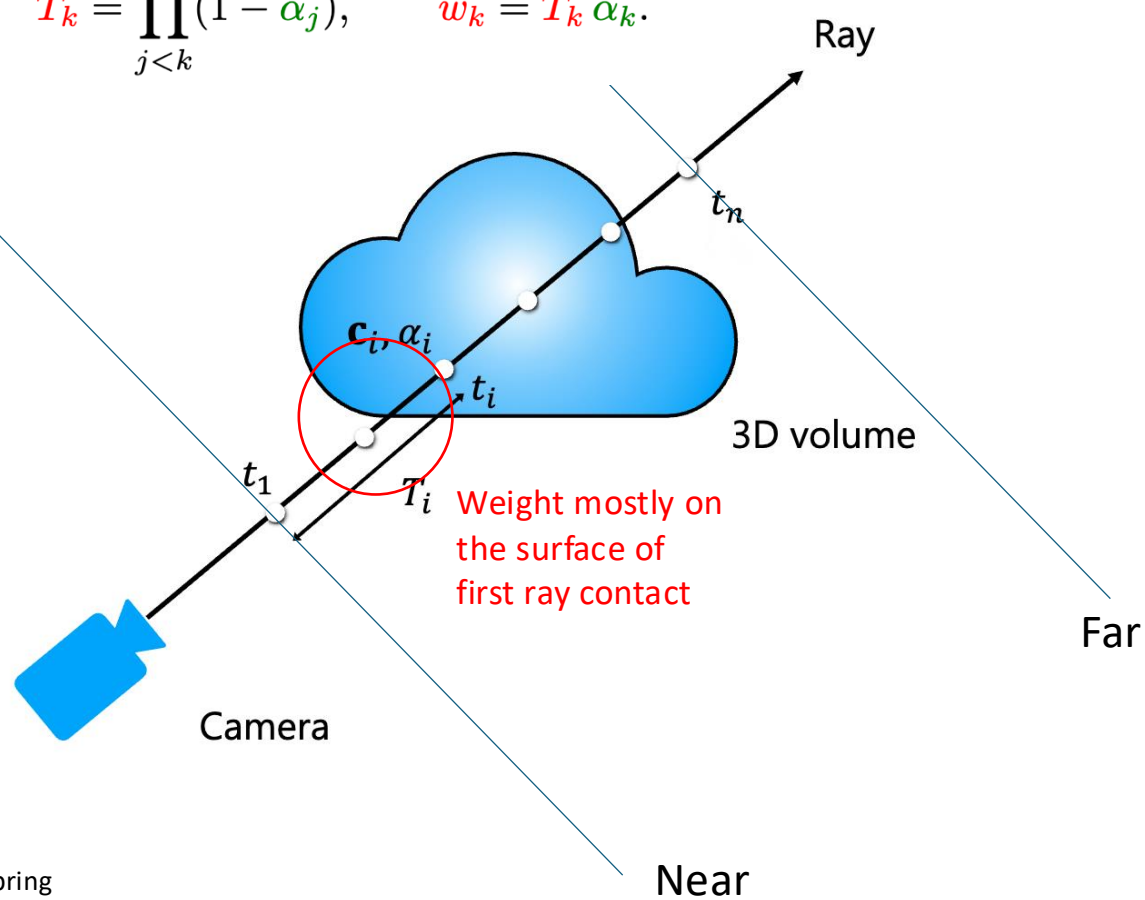


$$\alpha_k = 1 - \exp(-\sigma_k \, \delta_k), \qquad T_k = \prod_{j<k} (1 - \alpha_j), \qquad w_k = T_k \, \alpha_k.$$

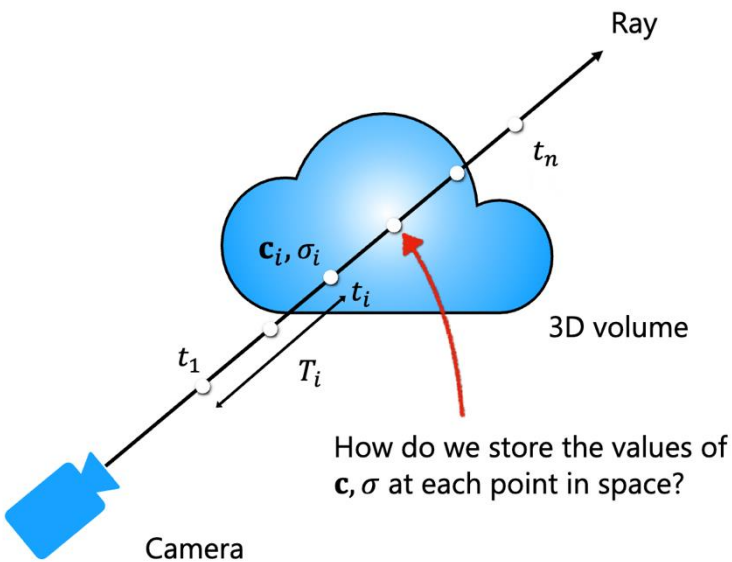
Volume Rendering

$$\alpha_k = 1 - \exp(-\sigma_k \, \delta_k), \qquad T_k = \prod_{j<k} (1 - \alpha_j), \qquad w_k = T_k \, \alpha_k.$$

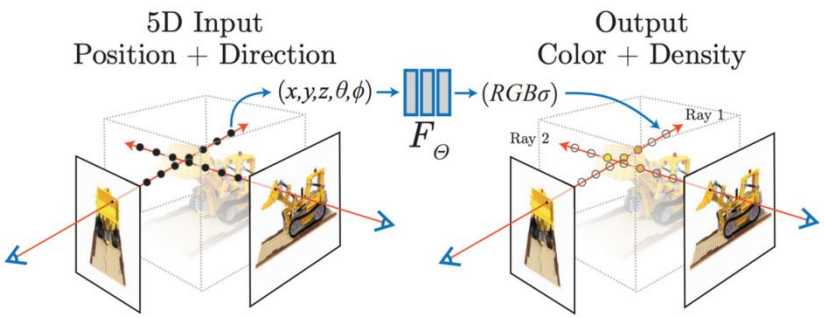
$$C \approx \sum_{k=1}^{K} w_k \, c_k$$



Volume Rendering



NeRF uses a MLP to model the function



From NeRF to Gaussian Splatting

- NeRF needs sampling allow the ray
- Gaussian allows for:
  - Faster Rendering
  - Explicit representation

Gaussian Splatting

Model radiance field with Gaussians

Represent the scene as a set of N Gaussians. For Gaussian i, store parameters

$$\{\mu_i \in \mathbb{R}^3,\ \Sigma_i \in \mathbb{R}^{3\times 3},\ \alpha_i \in (0,1),\ \boldsymbol{c}_i \in \mathbb{R}^3\}.$$

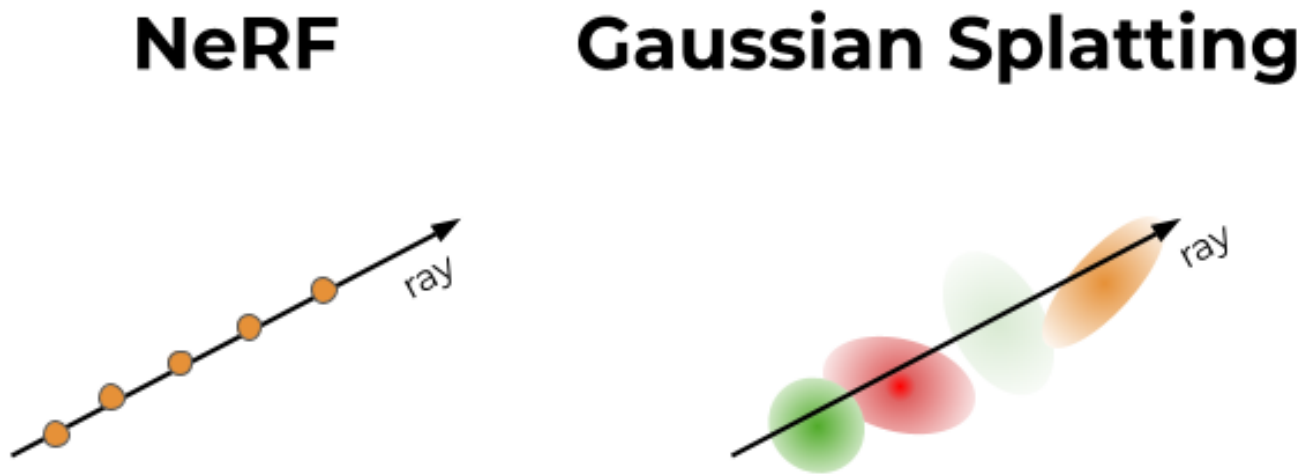
- μ_i : where the primitive lives in 3D
- Σ_i : anisotropic extent and orientation (ellipsoidal footprint)
- α_i : opacity / "how much it blocks light"
- $\boldsymbol{c}_i$: appearance (RGB; optionally view-dependent)

Explicit and sparse!

Gaussians are 3D blobs

Think of a Gaussian as a soft "blob" around μ:

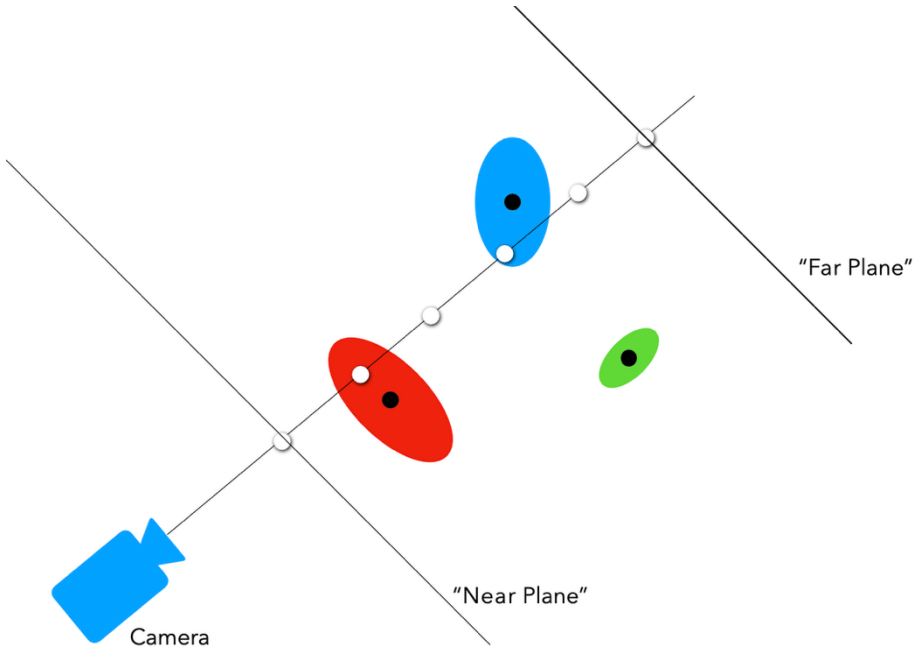
$$G(\boldsymbol{x}) \propto \exp\left(-\tfrac{1}{2}(\boldsymbol{x}-\mu)^\top \Sigma^{-1}(\boldsymbol{x}-\mu)\right).$$



Initialization: where do Gaussians come from?

- Randomly initialize Gaussian color and density
- Use SfM points to initialize the Gaussian centers

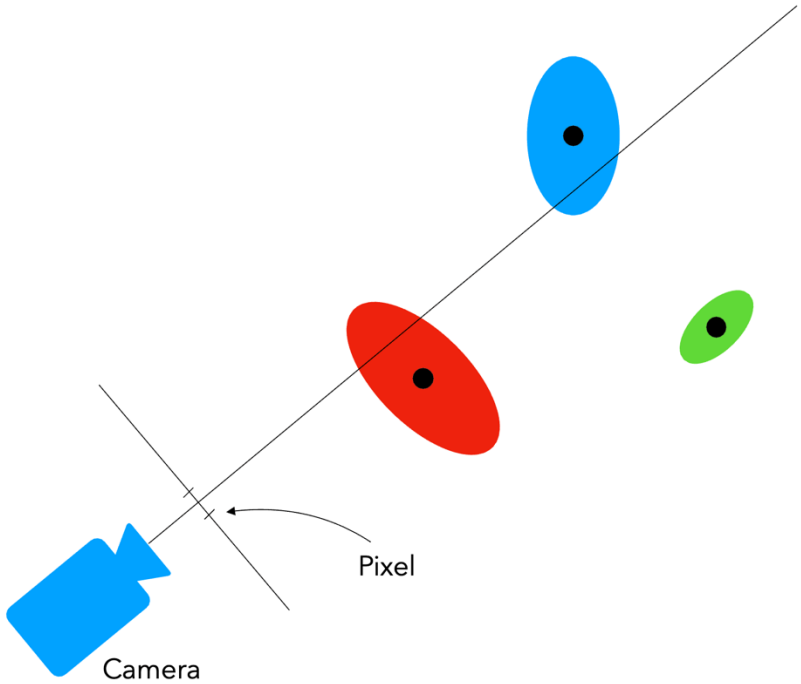
If we directly model radiance with Gaussians



Sampling too much empty space

Adapted from V. Sitzmann

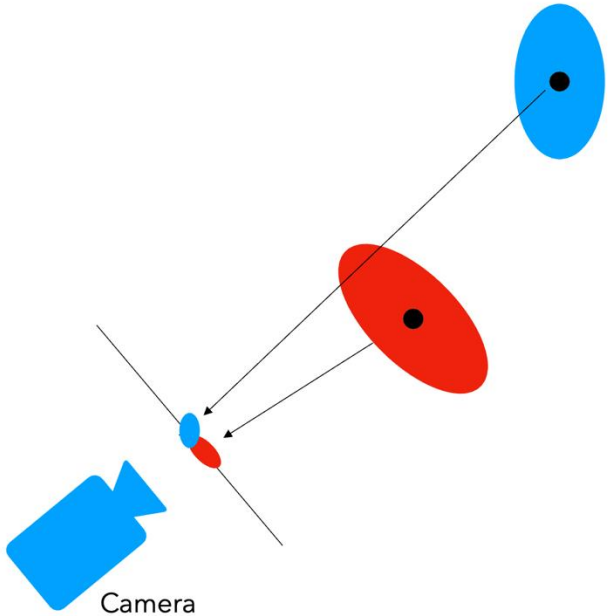
Instead: rasterization

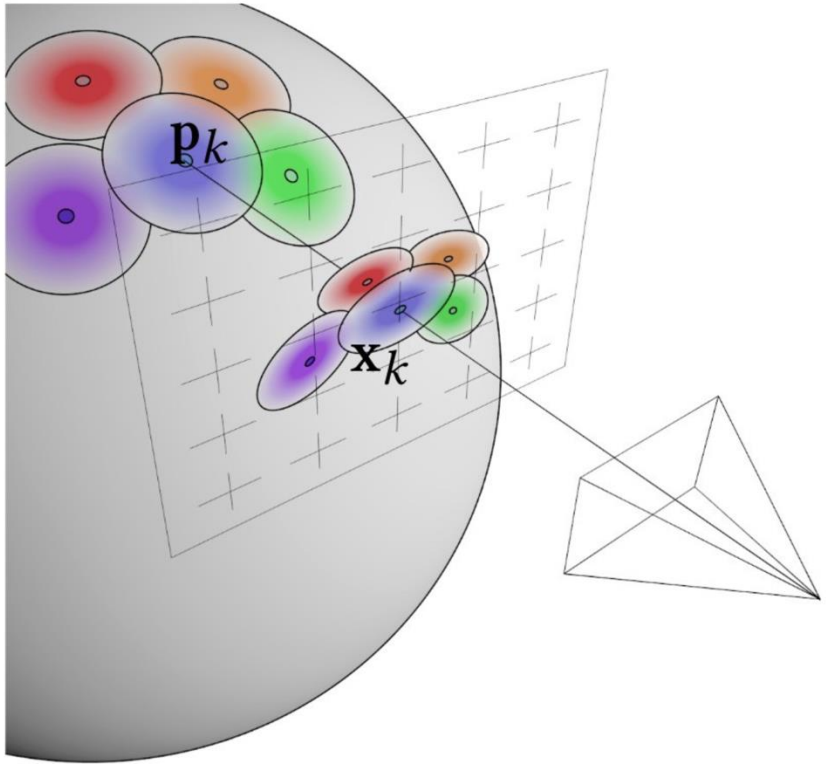


Project all Gaussians onto the image plane, and process each pixel.
Each pixel has all the relevant Gaussian information already.

Adapted from V. Sitzmann

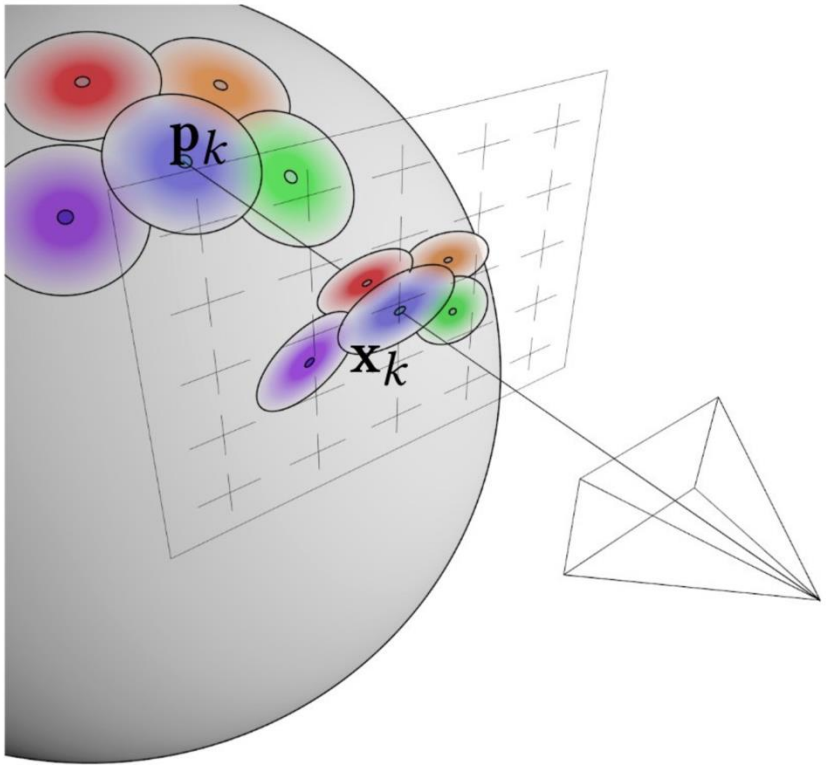
Rasterization

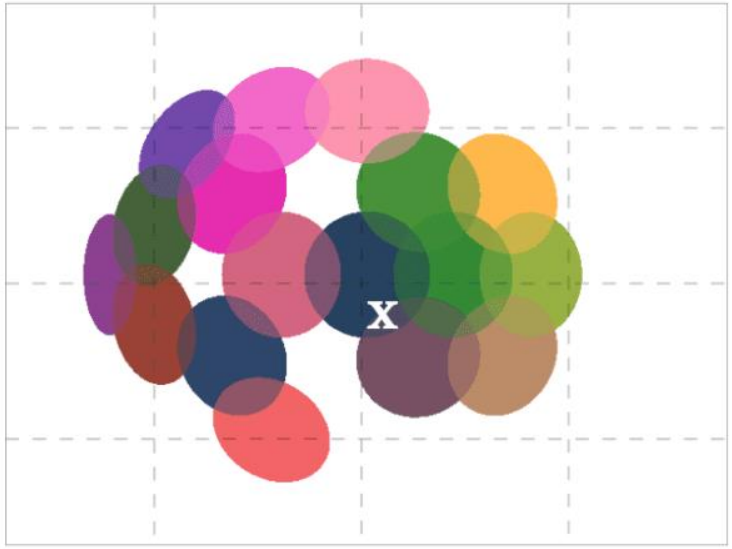




Adapted from V. Sitzmann

3D Gaussian to 2D Gaussian





Recap: 3D to 2D projection

For view v, map a 3D point in world coordinates to the camera frame:

$$\boldsymbol{x}_c = \mathbf{R}^{(v)}\,\boldsymbol{x}_w + \mathbf{t}^{(v)}.$$

Write $\boldsymbol{x}_c = (X, Y, Z)^\top$ with $Z > 0$. With intrinsics

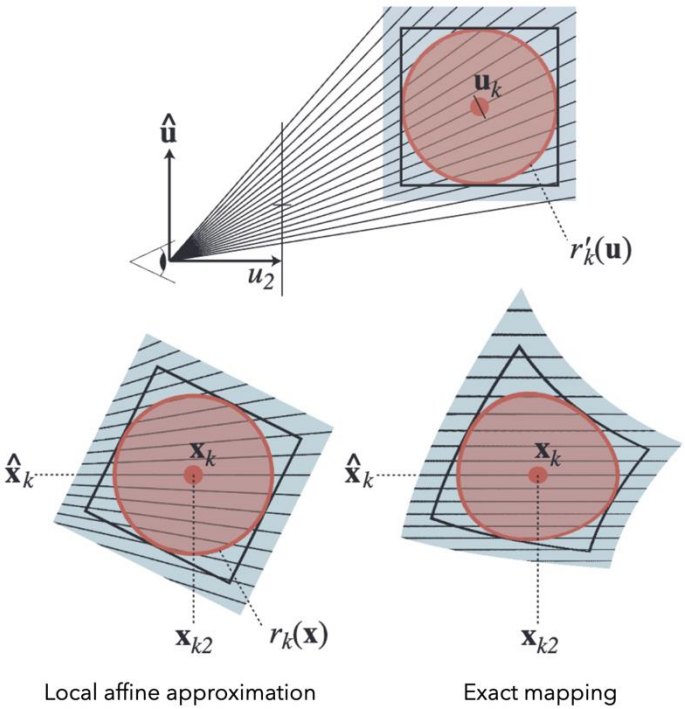
$$\mathbf{K} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

the pixel coordinates are

$$\boxed{u = f_x\,\frac{X}{Z} + c_x, \qquad v = f_y\,\frac{Y}{Z} + c_y}.$$

What does a projected Gaussian look like?

- Issue: a projected Gaussian is no longer a simple parametric shape



Solution: local affine approximation

Simplification: linearize Gaussian projection

$$\pi(\mu + \Delta \boldsymbol{x}) \approx \pi(\mu) + J\,\Delta \boldsymbol{x}, \qquad J = \left.\frac{\partial \pi}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}=\mu} \in \mathbb{R}^{2\times 3}.$$

Let $\mu = (X, Y, Z)^\top$ be the Gaussian mean in the *camera* frame and let $(u, v) = \pi(\mu)$. Using $u = f_x X/Z + c_x$ and $v = f_y Y/Z + c_y$, the Jacobian is

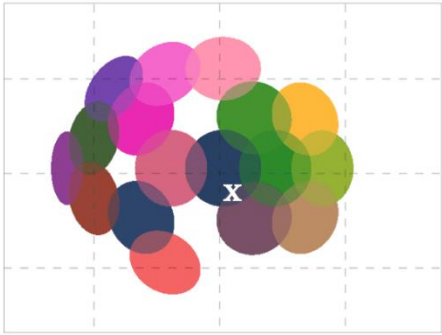
$$\boxed{J = \begin{bmatrix} \frac{\partial u}{\partial X} & \frac{\partial u}{\partial Y} & \frac{\partial u}{\partial Z} \\ \frac{\partial v}{\partial X} & \frac{\partial v}{\partial Y} & \frac{\partial v}{\partial Z} \end{bmatrix} = \left.\begin{bmatrix} \frac{f_x}{Z} & 0 & -\frac{f_x X}{Z^2} \\ 0 & \frac{f_y}{Z} & -\frac{f_y Y}{Z^2} \end{bmatrix}\right|_{(X,Y,Z)=\mu}}$$

We know that a Gaussian after a linear/affine transformation is still a Gaussian.
3D -> 2D

Affine transform of a Gaussian

Let $f : \mathbb{R}^3 \to \mathbb{R}^2$ and linearize around μ:

$$\boxed{f(\boldsymbol{x}) \approx f(\mu) + J(\boldsymbol{x} - \mu), \qquad J = \left.\frac{\partial f}{\partial \boldsymbol{x}}\right|_{\boldsymbol{x}=\mu} \in \mathbb{R}^{2\times 3}}$$



Define $\boldsymbol{Y} = f(\boldsymbol{X})$. Under the linear approximation,

$$\boldsymbol{Y} - \mathbb{E}[\boldsymbol{Y}] \approx J(\boldsymbol{X} - \mu).$$

So the covariance propagates as

$$\boxed{\mathrm{Cov}(\boldsymbol{Y}) \approx \mathrm{Cov}(J(\boldsymbol{X} - \mu)) = J\,\mathrm{Cov}(\boldsymbol{X})\,J^\top = J\Sigma J^\top}.$$

For splatting, take $f = \pi$ (camera projection), so $\Sigma_{2D} \approx J\Sigma_{3D}J^\top$.

Render step 1: calculate opacity alpha

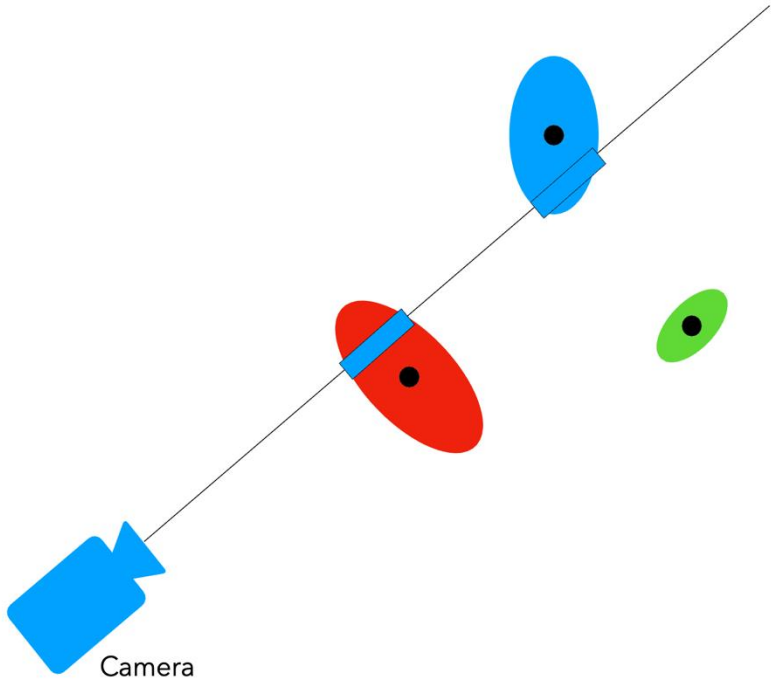
Define an unnormalized 2D weight at pixel p:

$$\boxed{w_i(p) = \exp\left(-\tfrac{1}{2}(p - u_i)^\top \Sigma_{2D,i}^{-1}(p - u_i)\right)}$$

Each Gaussian has a global opacity parameter α_i. At a pixel p, its effective opacity becomes

$$\boxed{\hat{\alpha}_i(p) = \alpha_i \, w_i(p)}.$$

Render step 2: alpha blending

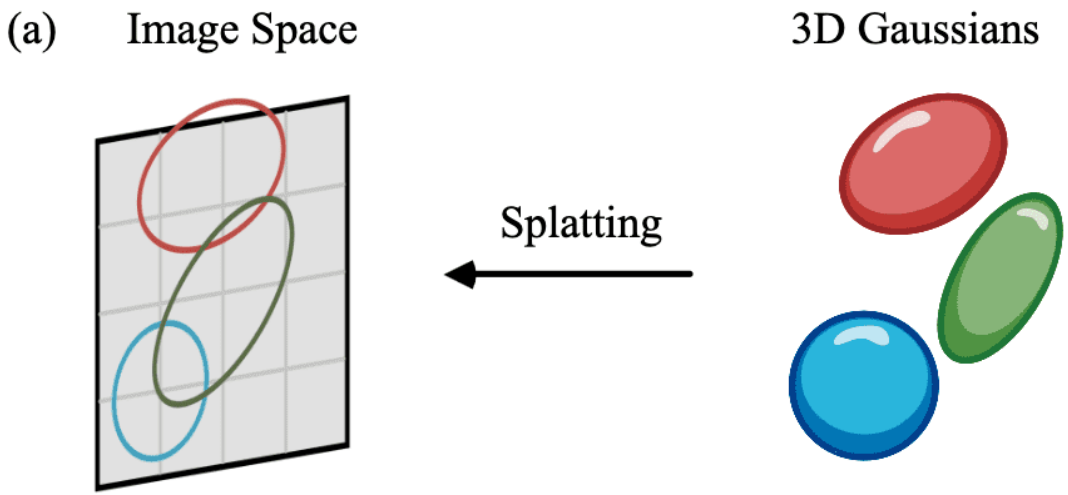


$$C = \sum_{i=1}^{N} \boldsymbol{c}_i \, \hat{\alpha}_i \prod_{j=1}^{i-1} (1 - \hat{\alpha}_j)$$

Adapted from V. Sitzmann

Book-keeping: tile-based rasterizer

- Each pixel only looks at Gaussians that overlap with a tile



Important: sort Gaussians first

- Order of Gaussians matters

$$C = \sum_{i=1}^{N} \boldsymbol{c}_i \, \hat{\alpha}_i \prod_{j=1}^{i-1} (1 - \hat{\alpha}_j)$$

- Need sort by depth in each tile
- Rasterization can be parallelized too

Compute loss for rendering

For each view v, render a prediction $\hat{I}^{(v)} = \mathrm{Render}_\theta(v)$.
Baseline photometric loss (L1):

$$\mathcal{L}_{\text{photo}} = \sum_v \|\hat{I}^{(v)} - I^{(v)}\|_1.$$

Structure Similarity (SSIM)

Structural similarity (SSIM): for image patches x, y (e.g., local windows),

$$\mu_x = \mathbb{E}[x],\ \mu_y = \mathbb{E}[y], \qquad \sigma_x^2 = \mathbb{E}[(x-\mu_x)^2],\ \sigma_y^2 = \mathbb{E}[(y-\mu_y)^2], \qquad \sigma_{xy} = \mathbb{E}[(x-\mu_x)(y-\mu_y)].$$

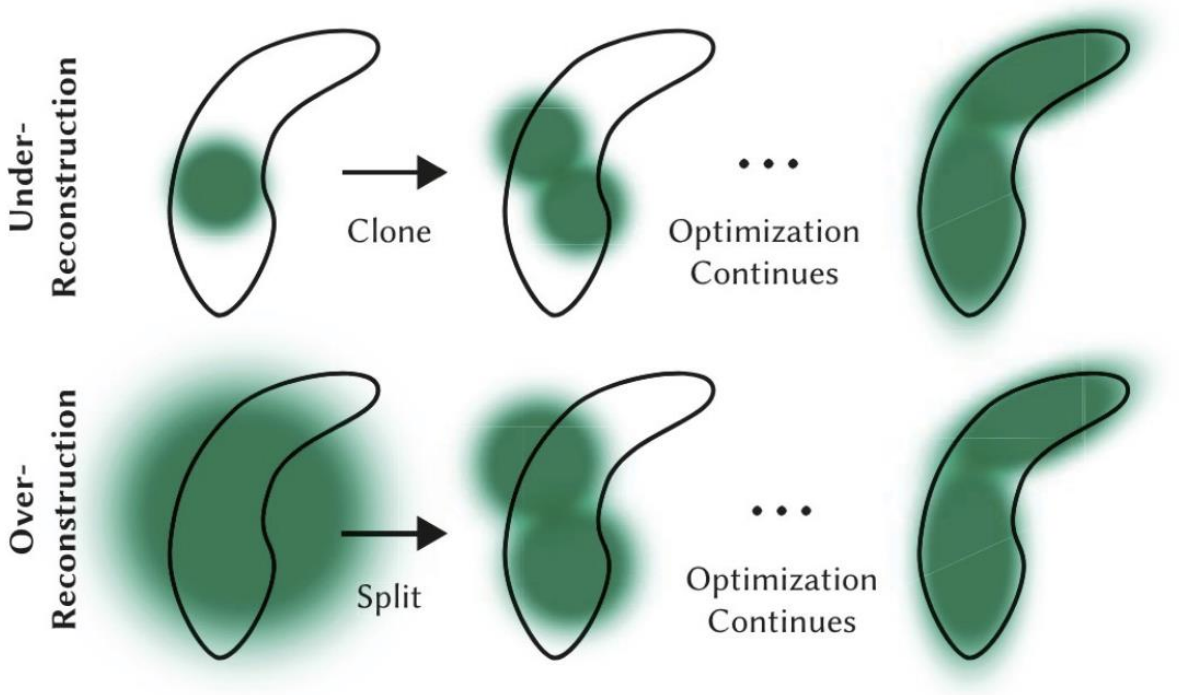
$$\boxed{\mathrm{SSIM}(x,y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}}$$

with constants $C_1 = (k_1 L)^2,\ C_2 = (k_2 L)^2$ (dynamic range L, typically $k_1 = 0.01,\ k_2 = 0.03$).

Summary statistics of the patches: similar brightness, similar variation, and correlation.

$$\mathcal{L} = (1-\lambda)\mathcal{L}_1 + \lambda\mathcal{L}_{\text{D-SSIM}}$$

Pruning and densification



Pruning

- If one Gaussian's opacity is too small
- If one Gaussian is extremely big

Densify

- If a Gaussian is big and the center has a high gradient, split it into smaller ones.
- If one Gaussian is small but the center has too much gradient, duplicate and move the new one towards the gradient.

Evaluation

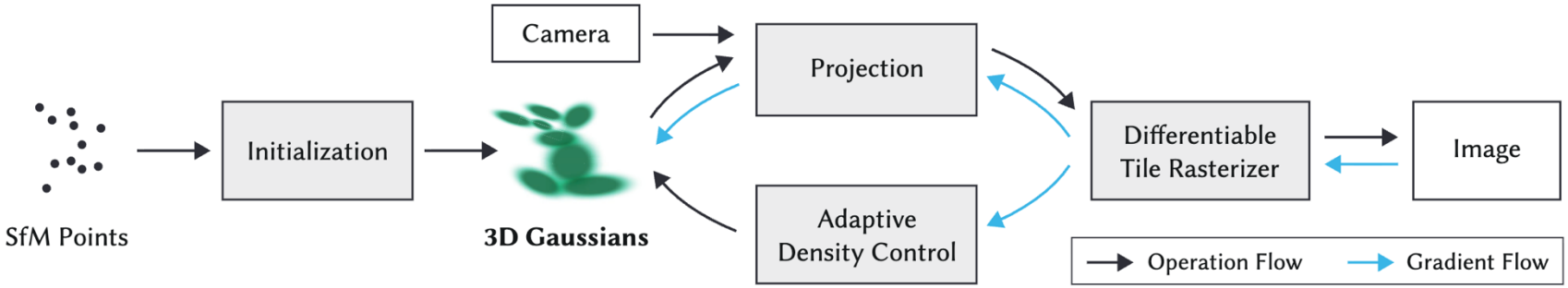
- PSNR

$$\mathrm{MSE}(I, \hat{I}) = \frac{1}{HW} \sum_{u,v} \left(I(u,v) - \hat{I}(u,v) \right)^2.$$

$$\mathrm{PSNR}(I, \hat{I}) = 10 \log_{10} \left(\frac{\mathrm{MAX}_I^2}{\mathrm{MSE}(I, \hat{I})} \right)$$

Higher PSNR is better; it is monotonically related to the L2/MSE reconstruction error.

Full Pipeline



Optimization Loop

Repeat until convergence:

1. pick one or more training views
2. render predicted images via splatting and compositing
3. compute loss (photometric + regularizers)
4. backpropagate gradients through the renderer
5. update parameters (e.g., Adam)
6. **(every M iterations)** densify and prune Gaussians (split high-error/large-footprint splats; remove negligible splats)

Example Gaussian Splatting

- https://antimatter15.com/splat/

HW2 Coding

- Given: pose, intrinsics, training views, initialization (point cloud)
- Training parameters: Gaussian parameters
- Output: test view images
- Evaluations: PSNR on test images